

Programmation événementielle et temps réel

Mounir BOUMIL

Titre et Introduction

- Introduction aux WebSockets et leur importance pour les applications en temps réel.
- Objectifs du cours :
 - Comprendre les bases des WebSockets.
 - Apprendre à mettre en place un serveur Node.js avec Socket.io.
 - Créer une application pratique : un chat en temps réel.

Qu'est-ce qu'un WebSocket ?

- **Définition :** Un protocole de communication bidirectionnelle full-duplex basé sur TCP.
- **Caractéristiques :**
 - Permet une communication en temps réel entre un client et un serveur.
 - Fonctionne sur une connexion unique, contrairement au HTTP classique qui nécessite une nouvelle connexion pour chaque requête.
- **Exemples d'utilisation :**
 - Chat en ligne.
 - Jeux multi-joueurs.
 - Applications de trading en temps réel.

La différence entre une communication HTTP classique (demande-réponse) et une communication via WebSocket (flux continu).

Comment fonctionnent les WebSockets ?

1. Établissement de la connexion :

- La communication commence par une requête HTTP classique avec un en-tête spécial pour « upgrader » la connexion en WebSocket.

2. Échange de données :

- Une fois la connexion établie, le client et le serveur peuvent envoyer des données à tout moment.

3. Fermeture de la connexion :

- La connexion peut être fermée par le client ou le serveur.

Installation des Outils Nécessaires

- **Node.js** : Plateforme permettant d'exécuter du JavaScript côté serveur.
 - Téléchargez et installez Node.js depuis nodejs.org.
- **NPM** : Gestionnaire de paquets intégré à Node.js pour installer des bibliothèques et dépendances.
- **Socket.io** : Bibliothèque pour implémenter facilement des WebSockets dans une application Node.js.

Commandes d'installation :

```
npm init -y          # Initialise un projet Node.js
npm install express socket.io # Installe Express et Socket.io
```

Création d'un Serveur Express de Base

- **Express** : Un framework web minimaliste pour Node.js, utilisé pour gérer les requêtes HTTP.
- **Code d'initialisation** :

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(3000, () => {
  console.log('Serveur à l\'écoute sur le port 3000');
});
```

Explication :

- Ce code crée un serveur qui répond "Hello World!" sur la route principale `/`.
- Le serveur écoute les requêtes sur le port 3000.

Introduction à Socket.io

- **Simplification des WebSockets** : Socket.io facilite l'utilisation des WebSockets en gérant les connexions, la compatibilité entre navigateurs, et les retours à d'autres méthodes si nécessaire (long polling).
- **Fonctionnalités supplémentaires** :
 - Gestion des événements.
 - Salles de discussion (rooms).
 - Prise en charge de la reconnexion automatique.

Mise en Place d'un Serveur Socket.io

- **Code d'initialisation :**

```
const http = require('http');
const { Server } = require('socket.io');

const server = http.createServer(app);
const io = new Server(server);

io.on('connection', (socket) => {
  console.log('Un utilisateur est connecté');
});

server.listen(3000, () => {
  console.log('Serveur à l\'écoute sur le port 3000');
});
```

Explication :

- `http.createServer(app)` : Crée un serveur HTTP en utilisant l'application Express.
- `new Server(server)` : Attache Socket.io au serveur HTTP.
- `io.on('connection', callback)` : Écoute les nouvelles connexions des clients.

Gestion des Messages avec Socket.io

- Écouter et émettre des messages :

```
io.on('connection', (socket) => {  
  console.log('Un utilisateur est connecté');  
  
  socket.on('chat message', (msg) => {  
    io.emit('chat message', msg);  
  });  
  
  socket.on('disconnect', () => {  
    console.log('Un utilisateur est déconnecté');  
  });  
});
```

Explication :

- `socket.on('chat message', callback)` : Écoute les messages envoyés par le client.
- `io.emit('chat message', msg)` : Diffuse le message à tous les clients connectés.

Créer une Interface Utilisateur

- Code HTML de base :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Chat en Temps Réel</title>
  </head>
  <body>
    <ul id="messages"></ul>
    <form id="form" action="">
      <input id="input" autocomplete="off" /><button>Envoyer</button>
    </form>

    <script src="/socket.io/socket.io.js"></script>
    <script>
      var socket = io();
      document.getElementById('form').addEventListener('submit', function(e) {
        e.preventDefault();
        socket.emit('chat message', document.getElementById('input').value);
        document.getElementById('input').value = '';
      });
      socket.on('chat message', function(msg) {
        var item = document.createElement('li');
        item.textContent = msg;
        document.getElementById('messages').appendChild(item);
      });
    </script>
  </body>
</html>
```

Tester et Déployer

- **Test Local :**

- Démarrez le serveur : `node server.js`.
- Accédez à `http://localhost:3000` dans plusieurs onglets ou navigateurs pour tester le chat en temps réel.

- **Déploiement :**

- Options de déploiement sur des plateformes comme Heroku, Vercel, ou tout autre hébergeur compatible Node.js.
- Configurez les variables d'environnement et les ports pour le déploiement.